Chapter 19 | Exercise solutions with Python

Translation and Summarization

This notebook solves Exercises 5–8. It begins with the rows an encoder–decoder receives during training, follows candidate paths at generation time, and compares the same candidate and reference under several overlap measures.

The examples are small enough to check by hand. Each section derives the result first, calls the chapter implementation, and uses assertions to compare the observed output with the expected result. The written solutions cover Exercises 1–4 and 9–11.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch19_generation"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Inspect a padded batch

Pair source `[3, 4]` with target `[5, 7, 6]` and source `[5, 6, 7, 8, 9]` with target `[8, 9]`. The encoder reads the source rows. The decoder reads target tokens shifted one position to the right of the corresponding labels. It starts with `BOS = 1` and learns to predict the end marker `EOS = 2` after the final content token.

The longest source has five tokens, so both source rows have five columns after padding. The longest target has three content tokens. Each of the two target-side arrays needs four columns, one for each content token plus one for its boundary marker. The batch has two rows in all three arrays, and `PAD = 0` fills any unused columns on the right.

```
from examples.ch19_generation.model import PAD, collate_pairs

pairs = [
    ([3, 4], [5, 7, 6]),
    ([5, 6, 7, 8, 9], [8, 9]),
]
source_ids, decoder_ids, labels = collate_pairs(pairs)

np.testing.assert_array_equal(
    source_ids, [[3, 4, 0, 0, 0], [5, 6, 7, 8, 9]]
)
np.testing.assert_array_equal(
    decoder_ids, [[1, 5, 7, 6], [1, 8, 9, 0]]
)
np.testing.assert_array_equal(
    labels, [[5, 7, 6, 2], [8, 9, 2, 0]]
)

print("Source IDs", source_ids.shape)
print(source_ids)
print("Decoder input IDs", decoder_ids.shape)
print(decoder_ids)
print("Labels", labels.shape)
print(labels)
```

OUTPUT

```
Source IDs (2, 5)
[[3 4 0 0 0]
 [5 6 7 8 9]]
Decoder input IDs (2, 4)
[[1 5 7 6]
 [1 8 9 0]]
Labels (2, 4)
[[5 7 6 2]
 [8 9 2 0]]
```

A label is valid when its ID differs from `PAD`. For example, the second decoder row reads `9` in the third column and must predict `EOS` there. That position contributes to the loss, while the fourth column in the same row is padding. The loss counts all five content labels and both end markers.

```
valid_labels = labels != PAD
expected_mask = [
    [True, True, True, True],
    [True, True, True, False],
]
np.testing.assert_array_equal(valid_labels, expected_mask)

valid_per_row = valid_labels.sum(axis=1)
np.testing.assert_array_equal(valid_per_row, [4, 3])
assert valid_labels.sum() == 7

print("Loss mask")
print(valid_labels)
print("Valid labels per row", valid_per_row)
print("Valid labels in the batch", valid_labels.sum())
```

OUTPUT

```
Loss mask
[[ True  True  True  True]
 [ True  True  True False]]
Valid labels per row [4 3]
Valid labels in the batch 7
```

6. Stop at a length cap

The first call to the probability function receives the empty tuple. This means the search has generated zero tokens. The only first choice is A, whose ID is 3. When the prefix is `(3,)`, the only next choice is the end-of-sequence token `EOS`, whose ID is 2.

A beam of width one keeps just the highest-scoring candidate at each step. One step generates A and uses up the budget. With two steps, the search can append `EOS`. The callback supplies a dictionary for each prefix that can be expanded in these runs. Once `EOS` is appended, the search considers that path complete and never asks for another token from it.

```
import math

from examples.ch19_generation.model import EOS, beam_search

A = 3

def capped_probabilities(prefix):
    probabilities = {(): {A: 1.0}, (A,): {EOS: 1.0}}
    return probabilities[prefix]

expected_paths = [(1, (A,), False), (2, (A, EOS), True)]
print(f"{'Max steps':>9}  {'Tokens':<10}  {'Finished':<8}  Log P")
for max_steps, expected_tokens, expected_finished in expected_paths:
    tokens, log_p, finished = beam_search(
        capped_probabilities, beam_width=1, max_steps=max_steps
    )
    print(f"{max_steps:>9}  {str(tokens):<10}  {str(finished):<8}"
          f"  {log_p:.1f}")
    assert tokens == expected_tokens
    assert finished == expected_finished
    assert math.isclose(log_p, 0.0)
```

```
OUTPUT
Max steps  Tokens      Finished  Log P
        1  (3,)        False     0.0
        2  (3, 2)      True      0.0
```

At the one-step cap, the return value is `((3,), 0.0, False)`. At the two-step cap, it is `((3, 2), 0.0, True)`. The second result includes the end token because emitting it takes a step. Both log-probabilities are zero. The one-token prefix has probability 1, and multiplying by the conditional probability 1 of `EOS` gives the same probability for the complete sequence. The flag records whether the returned tokens end in `EOS`.

7. Compare overlap measures

The candidate is *red red ferry departs* and the reference is *red ferry departs at noon*. An n-gram is a run of *n* adjacent tokens. A unigram is one token. For each unigram, clipping takes the smaller of its counts in the two sequences. The candidate has two occurrences of `red`, and the reference has one, so `red` contributes one match. The other matches are `ferry` and `departs`, for three in total.

A bigram is a pair of adjacent tokens in order. The candidate has `red red`, `red ferry`, and `ferry departs`. The reference has `red ferry`, `ferry departs`, `departs at`, and `at noon`. The two matching bigrams are `red ferry` and `ferry departs`.

Swapping the inputs exchanges the two counts for each n-gram. Taking their minimum gives the same result, so each clipped match total stays fixed. Precision divides by the number of n-grams in whichever sequence is now the candidate. The expected unigram precision goes from 3/4 to 3/5, and the expected bigram precision goes from 2/3 to 2/4.

```
from examples.ch19_generation.model import (
    clipped_ngram_precision,
    rouge_l,
)

candidate = "red red ferry departs".split()
reference = "red ferry departs at noon".split()

# Each row has the order, hand-counted matches, and denominators.
expected = [(1, 3, (4, 5)), (2, 2, (3, 4))]
for order, matches, expected_denominators in expected:
    denominators = (
        len(candidate) - order + 1,
        len(reference) - order + 1,
    )
    assert denominators == expected_denominators

    original = clipped_ngram_precision(candidate, reference, order)
    swapped = clipped_ngram_precision(reference, candidate, order)
    np.testing.assert_allclose(
        [original, swapped],
        [matches / total for total in expected_denominators],
    )
    print(f"Order {order}, denominators {denominators}")
    print(f"  Original {original:.6f}   Swapped {swapped:.6f}")
```

OUTPUT

```
Order 1, denominators (4, 5)
  Original 0.750000   Swapped 0.600000
Order 2, denominators (3, 4)
  Original 0.666667   Swapped 0.500000
```

ROUGE-L counts the length of a longest common subsequence. A subsequence keeps tokens in order and may skip intervening tokens. Here `red ferry departs` is common to both sequences. It starts at the second `red` in the candidate and the first `red` in the reference. It has length three, which is as long as any common subsequence can be here. Only three token occurrences can match even if we ignore their order, as the clipped unigram count showed.

Precision divides that length by the candidate length, and recall divides it by the reference length. The original precision is $3/4$ and recall is $3/5$. Swapping the inputs exchanges those values. F1 is the harmonic mean of precision and recall. Here it equals twice the common subsequence length divided by the sum of the two sequence lengths. Those quantities stay fixed after the swap, so F1 stays at $2(3)/(4+5) = 2/3$.

```
expected_subsequence = ["red", "ferry", "departs"]
candidate_positions = [1, 2, 3]
reference_positions = [0, 1, 2]
assert np.all(np.diff(candidate_positions) > 0)
assert np.all(np.diff(reference_positions) > 0)
assert [candidate[i] for i in candidate_positions] == (
    expected_subsequence
)
assert [reference[i] for i in reference_positions] == (
    expected_subsequence
)

original = rouge_l(candidate, reference)
swapped = rouge_l(reference, candidate)
assert original["lcs_length"] == swapped["lcs_length"] == 3
np.testing.assert_allclose(
    [original["precision"], original["recall"], original["f1"]],
    [3 / 4, 3 / 5, 2 / 3],
)
np.testing.assert_allclose(
    [swapped["precision"], swapped["recall"], swapped["f1"]],
    [original["recall"], original["precision"], original["f1"]],
)

print("One common subsequence", " ".join(expected_subsequence))
print("          LCS  Precision  Recall      F1")
for name, scores in [("Original", original), ("Swapped", swapped)]:
    print(
        f"{name:8} {scores['lcs_length']:4}  "
        f"{scores['precision']:9.6f}  {scores['recall']:.6f}  "
        f"{scores['f1']:.6f}"
    )
```

OUTPUT

```
One common subsequence red ferry departs
          LCS  Precision  Recall      F1
Original    3   0.750000  0.600000  0.666667
Swapped     3   0.600000  0.750000  0.666667
```

8. Keep a useful prefix (optional)

A starts with probability 0.65 and B starts with 0.35. With room for only one candidate, the decoder keeps A and discards B at the first step. A width of two keeps both. The score of each path is the product of its successive next-token probabilities. The implementation adds their logarithms and ranks candidates by decreasing log-probability, which gives the same order as ranking by decreasing probability.

The table in the callback gives every distribution that the four requested runs can query. An `EOS` candidate keeps its score and can still occupy a beam slot. The branch that ends in D can first be generated on step three, so the callback needs no next-token distribution after D for these step caps.

```
import math

from examples.ch19_generation.model import EOS, beam_search

A, B, C, D = 3, 4, 5, 6
token_names = {A: "A", B: "B", C: "C", D: "D", EOS: "EOS"}

def branching_probabilities(prefix):
    probabilities = {
        (): {A: 0.65, B: 0.35},
        (A,): {C: 0.7, EOS: 0.3},
        (B,): {EOS: 1.0},
        (A, C): {EOS: 0.6, D: 0.4},
    }
    return probabilities[prefix]
```

At step two, A can extend to A C with probability $0.65 \times 0.7 = 0.455$ or A EOS with probability $0.65 \times 0.3 = 0.195$. If B survived step one, it extends to B EOS with probability $0.35 \times 1 = 0.350$. Width one keeps A C. Width two also keeps B EOS, which outranks A EOS. In both runs with a two-step cap, the highest-scoring retained path is A C, so the search returns it with `finished=False`. A finished path can share the beam with a higher-scoring unfinished path.

```
expected = [
    (1, 2, (A, C), 0.455, False),
    (2, 2, (A, C), 0.455, False),
    (1, 3, (A, C, EOS), 0.273, True),
    (2, 3, (B, EOS), 0.350, True),
]

print("Width  Max steps  Path     Token IDs   Probability  Finished")
for width, steps, expected_tokens, expected_p, expected_flag in expected:
    tokens, log_p, finished = beam_search(
        branching_probabilities, beam_width=width, max_steps=steps
    )
    path = " ".join(token_names[token] for token in tokens)
    probability = math.exp(log_p)
    print(f"{width:>5}  {steps:>9}  {path:<7}  {str(tokens):<10}"
          f"  {probability:>11.3f}  {str(finished):>8}")
    assert tokens == expected_tokens
    assert math.isclose(probability, expected_p)
    assert finished == expected_flag
```

OUTPUT

```
Width  Max steps  Path     Token IDs   Probability  Finished
    1          2  A C      (3, 5)            0.455     False
    2          2  A C      (3, 5)            0.455     False
    1          3  A C EOS  (3, 5, 2)         0.273      True
    2          3  B EOS    (4, 2)            0.350      True
```

At step three, A C extends to A C EOS with probability $0.455 \times 0.6 = 0.273$ and A C D with probability $0.455 \times 0.4 = 0.182$. Width one returns A C EOS, the higher-scoring of the two extensions. With width two, B EOS carries forward at 0.350 without another probability factor. It ranks above both extensions of A C, so the two survivors in order are B EOS and A C EOS. The returned path ends in `EOS` at both widths with the three-step cap. B EOS can win only in the run that kept B at step one.